`HandleAsync` method that takes an `AuthorizationHandlerContext` object as a parameter and returns a `Task`. The logic for handling the requirement is placed in this method.

Requirements can have multiple handlers, but where there is a one-to-one relationship between the requirement and a handler, it is common to see the code for both placed in the same class, which implements both interfaces. The following example illustrates how to migrate the assertion you created earlier into such a class, which takes a parameter representing the number of months. Add a new folder to the project with the name AuthorizationRequirement, and then add a new class named `ViewRolesRequirement` with the code in the following listing.

Listing 10.19 A custom requirement with in-built handler

```
using Microsoft.AspNetCore.Authorization;

namespace CityBreaks.AuthorizationRequirements
{
    public class ViewRolesRequirement :
    ➥ IAuthorizationRequirement, IAuthorizationHandler                ❶
    {
        public int Months { get; }                                     ❷
        public ViewRolesRequirement(int months)                        ❷
        {                                                              ❷
            Months = months > 0 ? 0 : months;                          ❷
        }                                                              ❷

        public Task HandleAsync(                                       ❸
        ➥ AuthorizationHandlerContext context)                        ❸
        {

            var joiningDateClaim = context.User.                       ❹
            ➥ FindFirst(c => c.Type == "Joining Date")?.Value;        ❹
            if(joiningDateClaim == null)                               ❹
            {                                                          ❹
                return Task.CompletedTask;                             ❹
```